Отчет по лабораторной работе №5

Линейная регрессия с L1-регуляризацией: SIMD и OpenMP

Выполнила: Шакирова Асиля Ришатовна, 11-401

Параметры экспериментов и компьютер

Компьютер: Apple M1 Max, 10 ядер (8 производительных и 2 энергоэффективных), 64 ГБ памяти. Компилятор: Apple clang 17.0.0 с libomp, флаги -O3 -mcpu=native -fopenmp (-mcpu=native - аналог -march=native для Apple Silicon); в таблице 3.7 флаги сравниваются отдельно. Основные параметры: N = 10^5, d = 32, K = 100 итераций, η = 0,5, λ = 10^{-3}, 8 потоков, ε = 10^{-6}; в таблицах 3.3-3.6 меняется только исследуемый параметр, остальные указаны в заголовке таблицы. Данные: признаки равномерно из [-1; 1], seed = 42, y = Xw* + b* + шум, где истинные w* циклически равны 1; 0,05; 0,005; 0, b* = 0,5, шум до 0,1. Начальные w = 0, b = 0 у всех вариантов. Время - обучение K итераций без генерации данных и вывода, медиана 5 замеров. Допуск проверки корректности 10^{-8} (для Δw и для разности функций потерь). Каждая таблица получена отдельным запуском, поэтому время одной и той же конфигурации (N = 10^5, d = 32, 8 потоков) в таблицах отличается до 9%.

3.1. Корректность (N = 1000, d = 32)

Реализация	Функция потерь F	Δw	Корректность подтверждена?
Последовательная	0,009965748	0	да
OpenMP	0,009965748	$2{,}2\cdot10^{-16}$	да
SIMD	0,009965748	$2{,}2\cdot10^{-16}$	да
OpenMP + SIMD	0,009965748	$2{,}2\cdot10^{-16}$	да

3.2. Сравнение реализаций (N = 10^5, d = 32, 8 потоков)

Вариант	Время, с	Ускорение S
Последовательная	0,1530	1,00
OpenMP	0,0333	4,59
SIMD	0,1021	1,50
OpenMP + SIMD	0,0282	5,42

3.3. Число объектов N (d = 32)

N	serial, с	OpenMP, с	SIMD, с	OpenMP + SIMD, с
10000	0,0153	0,0105	0,0102	0,0088
100000	0,1527	0,0332	0,1019	0,0285
1000000	1,5281	0,2327	1,0221	0,2292

3.4. Число признаков d (N = 10^5)

d	serial, с	OpenMP, с	SIMD, с	OpenMP + SIMD, с
8	0,0353	0,0149	0,0353	0,0147
32	0,1526	0,0328	0,1034	0,0289
128	1,3298	0,1961	0,3918	0,0940
256	2,6822	0,3780	0,7580	0,1799

3.5. Потоки (N = 10^5, d = 32; вместо 16 взято 10)

p	OpenMP, с	S_p	E_p	OpenMP + SIMD, с	S_p	E_p
1	0,1548	1,00	1,00	0,1024	1,00	1,00
2	0,0816	1,90	0,95	0,0553	1,85	0,93
4	0,0466	3,32	0,83	0,0450	2,28	0,57
8	0,0325	4,76	0,60	0,0269	3,81	0,48
10	0,0402	3,85	0,38	0,0320	3,20	0,32

3.6. Коэффициент λ (N = 10^5, d = 32)

λ	F(w, b)	Число $\lvert w_j\rvert < \varepsilon$	Время (serial), с
0	0,001666	0	0,1552
10^{-4}	0,002510	5	0,1551
10^{-3}	0,010070	8	0,1553
10^{-2}	0,083302	16	0,1551
10^{-1}	0,685386	24	0,1551

3.7. Флаги компиляции (N = 10^5, d = 32, 8 потоков)

Флаги компиляции	Время, с	Ускорение относительно первой строки	Примечания
-O0 -fopenmp	0,2186	1,00	версия OpenMP; последовательная 1,453 с
-O3 -fopenmp	0,0337	6,50	последовательная 0,156 с
-O3 -mcpu=native -fopenmp	0,0325	6,72	последовательная 0,155 с
-O3 -mcpu=native -fopenmp + omp simd	0,0265	8,26	версия OpenMP + SIMD, быстрее версии OpenMP в 1,23 раза

Векторизация (отчет clang -Rpass=loop-vectorize вместо gcc -fopt-info-vec). Векторизованы (ширина 2 числа double) циклы по признакам j: скалярное произведение и накопление градиента во всех четырех вариантах (в simd-вариантах по директивам omp simd), а также шаг обновления w и расчет потерь. Не векторизованы циклы по итерациям K (внутри вызов memset), генерация данных (вызов rand()) и цикл проверки Δw (значение не распознано как редукция).

Флаги (3.7). Основной выигрыш дает -O3 (6,50 раза), -mcpu=native почти ничего не добавляет (6,72, на уровне разброса), omp simd поверх них дает 8,26.

Анализ результатов

1. Быстрее всех OpenMP + SIMD почти везде: при N = 10^5 0,0282 с (S = 5,4), при d = 256 0,180 с (S = 14,9). Исключения: при N = 1000 лучшей оказалась SIMD (0,0010 с против 0,0032 с у serial), а при N = 10^6 и при d = 8 OpenMP и OpenMP + SIMD почти не отличаются (0,233 и 0,229 с; 0,0149 и 0,0147 с). Влияние N (3.3): ускорение OpenMP растет с N: 1,46 при 10^4, 4,60 при 10^5, 6,57 при 10^6. Накладные расходы потоков (запуск и объединение копий g на каждой из K итераций) не зависят от N, поэтому на малых N они сравнимы с самой работой (при N = 1000 OpenMP даже медленнее serial), а на больших N становятся малы; до 8 ускорение не доходит, вероятно, из-за чтения данных из памяти (при N = 10^6 матрица X занимает 256 МБ). Влияние d (3.4): эффективность SIMD растет с d: ускорение SIMD относительно serial равно 1,0, 1,5, 3,4 и 3,5 при d = 8, 32, 128 и 256. Вероятно, при малом d вектор содержит мало элементов и накладные расходы цикла сравнимы с работой, а при большом d векторных операций много.

2. При 8 потоках S(OpenMP) = 4,59, S(OpenMP + SIMD) = 5,42; при d = 128 они равны 6,8 и 14,2: SIMD ускоряет вычисления в каждом потоке, поэтому общее ускорение может даже превышать число потоков. Ускорение OpenMP меньше числа потоков из-за последовательной части (обнуление и объединение g, шаг обновления), запуска и синхронизации потоков на каждой итерации, а также, вероятно, из-за общей памяти и ядер разной скорости.

3. Эффективность падает с ростом потоков: у OpenMP 0,95, 0,83, 0,60 при 2, 4, 8 потоках, у OpenMP + SIMD 0,93, 0,57, 0,48. Насыщение видно при 8 потоках, а при 10 время даже растет (у OpenMP от 0,0325 до 0,0402 с), вероятно, из-за двух энергоэффективных ядер: все потоки ждут самый медленный.

4. Автоматически векторизовались все внутренние циклы по j, в том числе в serial и OpenMP без директив; после добавления omp simd отчет компилятора не изменился, а время изменилось: SIMD быстрее serial в 1,50 раза при d = 32 и в 3,4 раза при d = 128. Вероятно, без директивы сложение в скалярном произведении должно идти строго по порядку, а omp simd reduction разрешает независимые частичные суммы. При d = 8 и при -O0 omp simd не ускоряет (1,0 и 0,93).

5. С ростом λ коэффициенты уменьшаются и обнуляются: число нулевых из 32 равно 0, 5, 8, 16, 24 при λ = 0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1} (8, 16 и 24 - это один, два и три из четырех типов признаков по 8 штук с истинными весами 0, 0,005 и 0,05). Функция потерь растет (от 0,0017 до 0,685): добавляется штраф $\lambda\Sigma|w_j|$ и ухудшается точность из-за смещения коэффициентов. Время обучения от λ не зависит.

6. Вероятно, ограничивают: при малых N накладные расходы потоков, при больших N чтение данных из памяти, при росте числа потоков последовательная часть итерации и разная скорость ядер, при малых d короткие векторы. Объединение частичных градиентов при d до 256, вероятно, мало, но отдельно не измерялось.